🗂️ Task Manager API

Production-Ready REST API — End-to-End

Technical Project Documentation | Luis Moreno | 2026

✅ CI Passing 🔒 Security: A 📊 Coverage: A 🚀 Live on Railway

🌐 **Live API:** https://lm-task-manager-api.up.railway.app
📖 **Swagger UI:** https://lm-task-manager-api.up.railway.app/swagger-ui/index.html
❤️ **Health Check:** https://lm-task-manager-api.up.railway.app/actuator/health
💻 **GitHub:** https://github.com/Luis-Morenoo/task-manager-api

1. Project Overview

> *I have professional experience with Java Spring Boot in enterprise environments, but wanted to challenge myself to build something completely from the ground up — no existing codebase to reference, no senior dev to ask. Just me, the docs, and a blank IntelliJ window.* 😅

The Task Manager API is a production-ready REST API that covers the complete lifecycle of a backend Java service. Built, tested, secured, containerized, deployed, and monitored. The kind of thing you'd see in enterprise environments, just without the 47 approval tickets to deploy it. 😂

The project deliberately mirrors patterns used in professional environments:

- Layered architecture: Controller → Service → Repository
- Environment-based configuration — no secrets in source code
- Automated CI/CD pipeline with quality gates on every push
- Security vulnerabilities identified and fixed via SonarCloud
- Redis caching to reduce database load on hot endpoints
- Health check endpoint for production monitoring

2. Technology Stack

Category	Technology	Purpose
Language & Framework	Java 21, Spring Boot 3.5.13	Core application runtime
Database	MongoDB Atlas	Cloud-hosted NoSQL document store
Caching	Redis (Railway)	In-memory cache — sub-millisecond reads
Build Tool	Gradle	Dependency management and build lifecycle
Containerization	Docker (multi-stage)	Reproducible builds and deployment packaging
Deployment	Railway	Cloud platform — auto-deploy on push
CI/CD	GitHub Actions	Automated build, test, and quality pipeline

Code Quality	SonarCloud	Static analysis — All A grades
API Docs	Swagger UI / OpenAPI 3.0	Interactive endpoint documentation
Testing	JUnit 5, Mockito, MockMvc	Unit and integration test layers
Logging	SLF4J with Logback	Structured application logging
Monitoring	Spring Boot Actuator	Health checks and metrics endpoints

3. Architecture

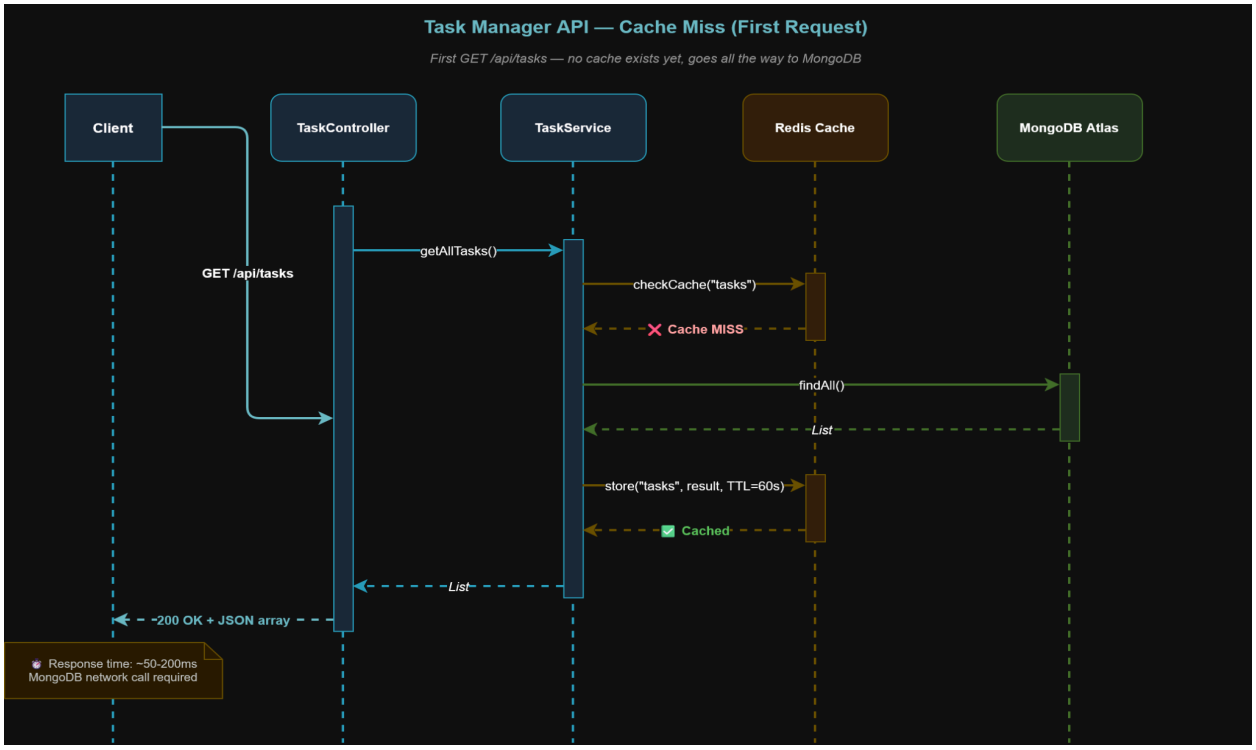
The API follows a clean layered architecture the same pattern used in enterprise Java development, except this time every layer was built from scratch. 😄

```
HTTP Request
    └→  Controller      (receives request, validates input, returns response)
            └→  Service        (business logic, cache management)
                    └→  Repository    (Spring Data MongoDB abstraction)
                            └→  MongoDB Atlas  (cloud database)
            ↕
        Redis Cache     (in-memory, TTL 60s)
```

3.1 Layer Responsibilities

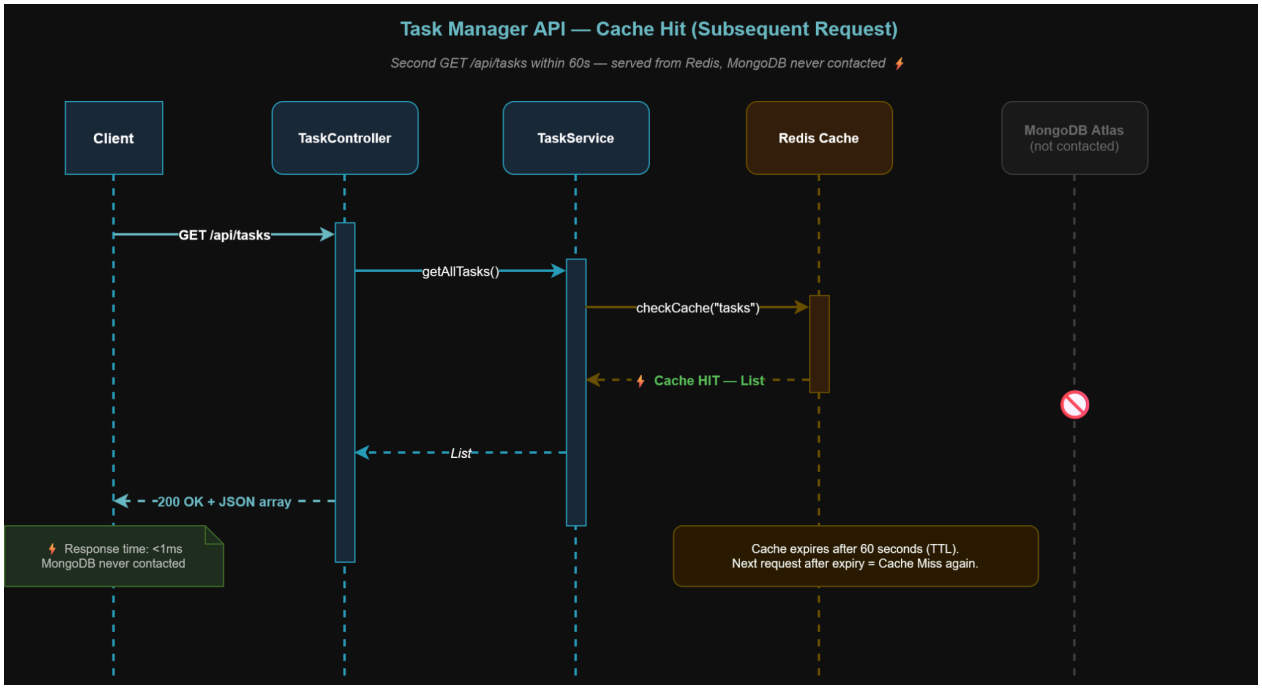
Layer	Responsibility
Controller	Handles HTTP request/response, input sanitization, log injection prevention
Service	Business logic, @Cacheable and @CacheEvict annotations, DTO-to-entity mapping
Repository	Extends MongoRepository — CRUD operations provided for free by Spring Data
Model	Task entity mapped to MongoDB documents via @Document annotation
DTO	TaskRequest — decouples request contract from the database entity (security)

Cache Miss - First request (hits MongoDB)



First request within a 60-second window — Redis has no cached result so the full call chain executes through to MongoDB

Cache Hit - Subsequent Request (served from Redis ⚡)



Subsequent requests within the TTL window — Redis returns the cached result instantly. MongoDB is never contacted

3.2 Caching Strategy

Redis sits alongside the service layer as a read-through cache for the GET all tasks endpoint. First request hits MongoDB and stores the result in Redis with a 60-second TTL. Subsequent requests return the cached result without touching the database.

Cache invalidation is triggered automatically on any write operation (create, update, delete) via @CacheEvict ensuring clients never receive stale data.

4. API Endpoints

Full interactive documentation is available on the live Swagger UI. All endpoints return JSON. Base URL: https://lm-task-manager-api.up.railway.app

Method	Endpoint	Auth	Description
GET	/api/tasks	None	Returns array of all tasks
GET	/api/tasks/{id}	None	Returns single task by MongoDB ObjectId
POST	/api/tasks	None	Creates new task from JSON body (TaskRequest DTO)
PUT	/api/tasks/{id}	None	Updates existing task fields by ID
DELETE	/api/tasks/{id}	None	Deletes task, returns 204 No Content
GET	/actuator/health	None	Spring Boot health check — MongoDB + Redis status

4.1 Request Body (POST / PUT)

POST and PUT endpoints accept a TaskRequest DTO deliberately excludes the id field to prevent mass assignment attacks:

```
{
    "title":       "Buy groceries",
    "description": "Milk, eggs, bread",
    "status":      "pending"
}
```

4.2 Response Body

All successful responses return the full Task entity including the server-generated MongoDB ObjectId:

```
{
    "id":          "69f114a70bd3c4b914a73ee7",
    "title":       "Buy groceries",
    "description": "Milk, eggs, bread",
    "status":      "pending"
}
```

5. Security

> *SonarCloud flagged several real vulnerabilities during development — all fixed. This is exactly what enterprise security scanning tools do on every pull request. Turns out those flags are there for good reason.* 😅

5.1 Mass Assignment Prevention (DTO Pattern)

Without a DTO, the POST endpoint would accept a Task entity directly as the request body. This means a client could supply an id field and potentially overwrite an existing database document a classic mass assignment attack.

The fix: TaskRequest DTO has no id field. The client physically cannot send one. The server always generates the ID.

```
// BEFORE — vulnerable
public Task createTask(@RequestBody Task task)  // Task has an id field

// AFTER — safe
public Task createTask(@RequestBody TaskRequest taskRequest)  // No id field
```

5.2 Log Injection Prevention

User-controlled input (path variables, request body fields) is sanitized before being written to logs. Without sanitization, a malicious request could inject fake log entries by embedding newline characters in the URL.

```
// Sanitize before logging — prevent log injection (SonarCloud S5145)
String sanitizedId = id.replaceAll("[\n\r\t]", "_");
log.info("GET /api/tasks/{} - Fetching task by ID", sanitizedId);
```

```
// Use original value for database lookup — sanitized value would break queries
return taskService.getTaskById(id);
```

5.3 PATH Hijacking Prevention

System commands use absolute paths instead of relying on the PATH environment variable. An attacker who can modify PATH could substitute a malicious executable in place of cmd.exe. Using the absolute path eliminates the attack surface entirely.

```
// UNSAFE — relies on PATH
Runtime.getRuntime().exec(new String[]{"cmd", "/c", "start", url});

// SAFE — absolute path, non-writable system directory
Runtime.getRuntime().exec(new String[]{"C:\\Windows\\System32\\cmd.exe", "/c",
"start", url});
```

5.4 Secrets Management

All credentials are managed through environment variables never hardcoded in source code. Configuration files containing secrets are excluded from git via .gitignore.

Secret	Variable Name	Where It Lives
MongoDB connection string	MONGODB_URI	Local: application.yaml (gitignored)CI: GitHub Actions secretProd: Railway service variable
Redis connection URL	REDIS_URL	Local: application.yaml (gitignored)CI: GitHub Actions secretProd: Railway auto-injected

6. Testing

Two layers of automated tests because one layer is not enough and zero layers is how you find out about bugs in production. 😅

6.1 Unit Tests — TaskServiceTest

Mockito mocks out the repository entirely. No database, no network, no Spring context. Tests run in milliseconds and verify every service method in complete isolation using the Arrange-Act-Assert pattern.

Test Method	What It Verifies
shouldReturnAllTasks()	getAllTasks() returns correct list from mocked repository
shouldReturnTaskById()	getTaskById() returns correct Optional<Task> for matching ID
shouldCreateTask()	createTask() calls repository.save() and returns the saved entity
shouldUpdateTask()	updateTask() sets the ID before saving, returns updated entity
shouldDeleteTask()	deleteTask() calls repository.deleteById() — verified via Mockito.verify()

6.2 Integration Tests — TaskControllerIntegrationTest

Spring Boot Test with @SpringBootTest and MockMvc. Fires real HTTP requests through the full stack against a live MongoDB Atlas instance. Each test starts with a clean database via @BeforeEach deleteAll().

Test Method	What It Verifies
shouldCreateAndRetrieveTask()	POST creates task, GET all returns it — full roundtrip
shouldGetTaskById()	GET /{id} returns correct task from live database
shouldReturn404WhenTaskNotFound()	GET with fake ID returns 404 Not Found — covers warn log branch
shouldUpdateTask()	PUT updates fields, response reflects changes
shouldDeleteTask()	DELETE returns 204, subsequent GET returns 404 — verifies actual removal
shouldIgnoreClientProvidedId()	POST with id in body returns server-generated ID — proves mass assignment fix works

6.3 Running Tests

```
# Run all tests
./gradlew test

# Run with JaCoCo coverage report
./gradlew test jacocoTestReport

# Coverage report opens at:
# build/reports/jacoco/test/html/index.html
```

7. CI/CD Pipeline

Every push to main triggers the full automated pipeline. No manual deployments, no 'works on my machine' moments. Same concept as enterprise CI/CD pipelines automated quality gates before anything reaches production. 💪

```
Push to main
    │
    ├─→  GitHub Actions
    │         ├─  Checkout code (full git history for SonarCloud)
    │         ├─  Set up JDK 21 (Temurin distribution)
    │         ├─  Grant gradlew execute permission
    │         └─  ./gradlew build jacocoTestReport sonar
    │                   ├─  Compiles application
    │                   ├─  Runs all 7 tests (unit + integration)
    │                   ├─  Generates JaCoCo XML coverage report
    │                   └─  Sends results to SonarCloud for analysis
    │
    └─→  Railway (on successful build)
              ├─  Detects Dockerfile in repo root
              ├─  Builds Docker image (multi-stage)
              └─  Deploys container to us-west-2
```

7.1 GitHub Actions Secrets

Secret	Purpose
MONGODB_URI	Live Atlas connection for integration tests
REDIS_URL	Public Railway Redis URL for test environment
SONAR_TOKEN	Authentication token for SonarCloud analysis
SONAR_PROJECT_KEY	SonarCloud project identifier
SPRING_CACHE_TYPE	Set to 'none' — disables Redis during integration tests

8. Docker — Multi-Stage Build

The Dockerfile uses a two-stage build pattern a common production practice that keeps the final image lean and secure.

```
# Stage 1: Build environment — full JDK needed to compile
FROM eclipse-temurin:21-jdk-alpine AS build
WORKDIR /app
COPY gradlew .
COPY gradle gradle
COPY build.gradle .
COPY settings.gradle .
COPY src src
RUN chmod +x gradlew
RUN ./gradlew bootJar --no-daemon -x test   # tests already ran in CI

# Stage 2: Runtime environment — only JRE needed to run the JAR
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/build/libs/*.jar app.jar   # only the JAR, nothing else
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

> *Stage 1 uses JDK (~300MB) to compile. Stage 2 uses JRE (~80MB) to run. The final image is 80MB instead of 300MB+ — no compiler, no source code, no Gradle cache. Smaller image = faster deploys, smaller attack surface.*

8.1 Running with Docker

```
# Build the image
docker build -t task-manager .

# Run the container
docker run -p 8080:8080 \
  -e MONGODB_URI="your_atlas_uri_here" \
  -e REDIS_URL="your_redis_url_here" \
  task-manager
```

9. Code Quality — SonarCloud

SonarCloud performs static analysis on every CI run, evaluating four quality dimensions. All currently rated A.

Dimension	Grade	What It Measures
Security	A	Vulnerabilities, hardcoded secrets, injection risks
Reliability	A	Bugs, null pointer risks, unreachable code
Maintainability	A	Code smells, complexity, duplication
Coverage	A	Percentage of code executed by automated tests

Vulnerabilities fixed during development:

- S4684 — Mass assignment via entity in request body (fixed with DTO pattern)
- S5145 — Log injection via unsanitized user input (fixed with replaceAll sanitization)
- S4036 — PATH hijacking via relative command execution (fixed with absolute cmd.exe path)
- Hardcoded Redis URL in application.yaml (fixed with environment variable)

10. Monitoring & Health

Spring Boot Actuator provides production-ready health checks and metrics out of the box. Railway's Observability dashboard streams live application logs.

10.1 Health Check Response

GET /actuator/health returns the status of every application component:

```
{
  "status": "UP",
  "components": {
    "mongo": {
      "status": "UP",
      "details": { "maxWireVersion": 25 }
    },
    "diskSpace": { "status": "UP" },
```

```
    "ping":      { "status": "UP" },
    "ssl":       { "status": "UP" }
  }
}
```

> *The mongo: UP confirmation is the important one. It proves the app can actually reach its database — not just that the server is running. Railway is configured to ping this endpoint and auto-restart the container if it ever returns DOWN.*

10.2 Railway Observability

Railway streams container logs in real time through its Observability dashboard equivalent to viewing logs in an enterprise SIEM. Application logs flow through SLF4J/Logback with structured formatting:

```
2026-05-04 INFO  [task-manager] c.l.t.controller.TaskController : POST /api/tasks -
Creating new task: Buy groceries
2026-05-04 INFO  [task-manager] c.l.t.controller.TaskController : GET /api/tasks -
Fetching all tasks
2026-05-04 INFO  [task-manager] c.l.t.service.TaskService       : Fetching all
tasks from MongoDB
2026-05-04 WARN  [task-manager] c.l.t.controller.TaskController : GET
/api/tasks/fake123 - Task not found
```

11. Project Structure

```
task-manager-api/
├── .github/
│   └── workflows/
│       └── ci.yml                    # GitHub Actions pipeline
├── src/
│   ├── main/
│   │   ├── java/com/luis/taskmanager/
│   │   │   ├── controller/
│   │   │   │   └── TaskController.java       # HTTP endpoints, logging,
sanitization
│   │   │   ├── service/
│   │   │   │   └── TaskService.java          # Business logic, caching annotations
│   │   │   ├── repository/
│   │   │   │   └── TaskRepository.java       # MongoRepository extension
│   │   │   ├── model/
│   │   │   │   └── Task.java                 # MongoDB document entity
│   │   │   ├── dto/
│   │   │   │   └── TaskRequest.java          # Request body DTO (no id field)
```

```
│   │   │   ├── OpenApiConfig.java            # Swagger + auto-browser-open
│   │   │   └── TaskManagerApplication.java   # Spring Boot entry point
│   │   └── resources/
│   │           └── application.yaml.example      # Config template (gitignored: .yaml)
│   └── test/
│           ├── java/com/luis/taskmanager/
│           │   ├── TaskServiceTest.java           # Unit tests (Mockito)
│           │   └── TaskControllerIntegrationTest.java  # Integration tests (MockMvc)
│           └── resources/
│               └── application.yaml.example       # Test config template
├── Dockerfile                        # Multi-stage build
├── build.gradle                      # Dependencies + JaCoCo + SonarCloud plugins
└── README.md                         # Project documentation
```

12. Skills Demonstrated

Skill Area	Details
Backend Development	Java 21, Spring Boot 3.5, REST API design, layered architecture, dependency injection
Database	MongoDB Atlas, Spring Data MongoDB, document modeling, CRUD operations
Caching	Redis, Spring Cache abstraction, @Cacheable/@CacheEvict, TTL configuration
Security	DTO pattern, log injection prevention, PATH hijacking, env variable secrets management
Testing	JUnit 5, Mockito, MockMvc, unit + integration layers, security regression tests
CI/CD	GitHub Actions, automated test pipeline, SonarCloud integration, coverage reporting
Containerization	Docker, multi-stage builds, Alpine Linux images, container environment config
Deployment	Railway cloud platform, environment variables, health check wiring, auto-redeploy
Code Quality	SonarCloud static analysis, JaCoCo coverage, All A grades, vulnerability remediation
API Documentation	SpringDoc OpenAPI, Swagger UI, auto-generated interactive docs
Logging & Monitoring	SLF4J/Logback, structured logs, Spring Boot Actuator, Railway Observability
Build Tooling	Gradle, bootJar, jacocoTestReport, sonar task chaining

13. About the Developer

Luis Moreno

Computer Engineering Graduate | Software Engineer | Cybersecurity Graduate Student

📧 luismorenosofteng@gmail.com
💻 github.com/Luis-Morenoo
🎓 MS Computer Science — Cybersecurity @ Arizona State University (2026)
🌍 Chandler, AZ